Some more examples of scientific computation in C++

1. Extending the `MovingObject` Class

Question:

Modify the `MovingObject` class to support **different types of motion**, such as **free fall, projectile motion, and linear motion**. Use **inheritance** and **polymorphism** by creating a base class and derived classes.

Solution:

We define a **base class** **`MovingObject`** and extend it into **three derived classes**:

- `ProjectileMotion`
- `FreeFall`
- `LinearMotion`

Each derived class overrides the `position(double time)` function to calculate motion accordingly.

```
# include <iostream>
# include <cmath>
using namespace std;

// Base class
class MovingObject {
protected:
    double velocity;
    double angle; // Degrees
public:
    MovingObject(double v, double a) : velocity(v), angle(a) {}
    virtual double position(double time) = 0; // Pure virtual function
};

// Derived class for projectile motion
class ProjectileMotion : public MovingObject {
public:
    ProjectileMotion(double v, double a) : MovingObject(v, a) {}
    double position(double time) override {
        return velocity * sin(angle * M_PI / 180.0) * time - 0.5 * 9.81 * time * time;
    }
};

// Derived class for free fall
class FreeFall : public MovingObject {
public:
    FreeFall(double v) : MovingObject(v, 90) {}
    double position(double time) override {
        return velocity * time - 0.5 * 9.81 * time * time;
    }
};

int main() {
    MovingObject* obj1 = new ProjectileMotion(20, 45);
    MovingObject* obj2 = new FreeFall(0);

    cout << "Projectile Motion (t=2s): " << obj1->position(2) << " meters" << endl;
    cout << "Free Fall (t=2s): " << obj2->position(2) << " meters" << endl;

    delete obj1;
    delete obj2;
    return 0;
}
```

---

2. Overloading Operators for Motion Addition

Question:

Create a `Vector2D` class to represent **2D motion vectors** . Overload:

- The `+` and `-` operators to add or subtract vectors.
- A **dot product function** for finding dot product of vectors.

Solution:

```
# include <iostream>
using namespace std;

class Vector2D {
public:
    double x, y;
    Vector2D(double x_val = 0, double y_val = 0) : x(x_val), y(y_val) {}

    // Overload + operator
    Vector2D operator+(const Vector2D& other) {
        return Vector2D(x + other.x, y + other.y);
    }

    // Overload - operator
    Vector2D operator-(const Vector2D& other) {
        return Vector2D(x - other.x, y - other.y);
    }

    // Dot product
    double dotProduct(const Vector2D& other) {
        return x * other.x + y * other.y;
    }
};

int main() {
    Vector2D v1(3, 4), v2(1, 2);
    Vector2D sum = v1 + v2;
    Vector2D diff = v1 - v2;
    double dot = v1.dotProduct(v2);

    cout << "Sum: (" << sum.x << ", " << sum.y << ")\n";
    cout << "Difference: (" << diff.x << ", " << diff.y << ")\n";
    cout << "Dot Product: " << dot << "\n";
    return 0;
}
```

---

3. Storing Motion Data in Files

Question:

Modify a projectile motion program to store time vs. position data in a `.csv` file.

Solution:

```
# include <iostream>
# include <fstream>
using namespace std;

int main() {
    ofstream file("motion_data.csv");
    file << "Time,Position\n";

    for (double t = 0; t <= 5; t += 0.5) {
        double y = 20 * t - 0.5 * 9.81 * t * t;
        file << t << "," << y << "\n";
    }
    file.close();
    cout << "Data saved to motion_data.csv" << endl;
    return 0;
}
```

---

4. Simulating a Bouncing Ball

Question:

Modify the **projectile motion simulation** to simulate a **bouncing ball**, reversing velocity when it hits the ground.

Solution:

```
# include <iostream>
using namespace std;

void simulateBouncingBall(double velocity) {
    double time = 0, y;
    while (velocity > 0.5) {
        y = velocity * time - 0.5 * 9.81 * time * time;
        if (y < 0) {
            velocity *= 0.8; // Reduce velocity on impact
            time = 0; // Reset time counting
        }
        cout << "Time: " << time << " s, Height: " << y << " m\n";
        time += 0.1;
    }
}

int main() {
    simulateBouncingBall(20);
    return 0;
}
```

5.Practice Question:

Simulating Forces Acting on Objects

Question:

Create a base class called Force that represents a general force acting on an object. Derive classes for different types of forces such as:

- `GravitationalForce`
- `FrictionForce`
- `SpringForce (Hooke's Law)`

Each derived class should override a function calculateForce(double mass, double parameter) to compute the force based on its own formula.

Use polymorphism to call the force calculation function for different force types.

```
#include <iostream>
#include <cmath>

class Force {
public:

    double mass;
    double parameter;

    void setValues(double m, double p) {
        mass = m;
        parameter = p;
    }

    void calculate() { }
};

class GravitationalForce : public Force {
public:
    void calculate() {
        double result = mass * parameter;
        std::cout << "Gravitational Force: " << result << " N" << std::endl;
    }
};

class FrictionForce : public Force {
public:
```

```
    void calculate() {
        double normalForce = mass * 9.81;
        double result = parameter * normalForce;
        std::cout << "Friction Force: " << result << " N" << std::endl;
    }
};

class SpringForce : public Force {
public:
    void calculate() {
        double k = 10.0;
        double result = -k * parameter;
        std::cout << "Spring Force: " << result << " N" << std::endl;
    }
};

int main() {
    GravitationalForce gf;
    gf.setValues(2.0, 9.81); // mass = 2 kg, gravity = 9.81
    gf.calculate();

    FrictionForce ff;
    ff.setValues(2.0, 0.5); // mass = 2 kg, coefficient = 0.5
    ff.calculate();

    SpringForce sf;
    sf.setValues(2.0, 0.3); // displacement = 0.3 m
    sf.calculate();

    return 0;
}
```